

Atlas Oracle 2

Security Assessment

CertiK Assessed on Jun 5th, 2026





CertiK Assessed on Jun 5th, 2026

Atlas Oracle 2

The security assessment was prepared by CertiK.

Executive Summary

TYPES

Oracle

ECOSYSTEM

EVM Compatible

METHODS

Formal Verification, Static Analysis

LANGUAGE

Solidity

TIMELINE

Preliminary comments published on 05/22/2026

Final report published on 06/05/2026

Vulnerability Summary



8

Total Findings

3

Resolved

0

Partially Resolved

5

Acknowledged

0

Declined

0 Centralization

Centralization findings highlight privileged roles & functions and their capabilities, or instances where the project takes custody of users' assets.

0 Critical

Critical risks are those that impact the safe functioning of a platform and must be addressed before launch. Users should not invest in any project with outstanding critical risks.

0 Major

Major risks may include logical errors that, under specific circumstances, could result in fund losses or loss of project control.

2 Medium

1 Resolved, 1 Acknowledged



Medium risks may not pose a direct risk to users' funds, but they can affect the overall functioning of a platform.

4 Minor

2 Resolved, 2 Acknowledged



Minor risks can be any of the above, but on a smaller scale. They generally do not compromise the overall integrity of the project, but they may be less efficient than other solutions.

2 Informational

2 Acknowledged



Informational errors are often recommendations to improve the style of the code or certain operations to fall within industry best practices. They usually do not affect the overall functioning of the code.

TABLE OF CONTENTS | ATLAS ORACLE 2

Audit Summary

Executive Summary

Vulnerability Summary

Codebase

Audit Scope

Approach & Methods

Review Notes

Out-of-Scope Components

Centralization Related Risk

Findings

AO2-01 : Inherited Standard Consumer Relies On A Single Hardcoded Oracle Signer

AO2-02 : Potential Dirty Data In `signer` Reading

AO2-04 : Unupdated Free Memory Pointer

AO2-05 : Duplicate Feed IDs Are Accepted And Can Resolve To Conflicting Oracle Values

AO2-06 : Zero Price Values Are Accepted As Valid Oracle Data

AO2-07 : Unchecked `feedIds`

AO2-03 : Empty Initial Signer Set Can Disable Oracle Consumption

AO2-08 : Clarification On Price Feed Extra Data Replay And Subscriber Access Control

Appendix

Disclaimer

CODEBASE | ATLAS ORACLE 2

Repository

- Private Codebase
- [oracle-atlas/pull-oracle-consumer](#)

Commit

[fae36318e0a5bdf48956b9a54aa4fd19fe20b587](#)

Audit Scope

The file in scope is listed in the appendix.

APPROACH & METHODS | ATLAS ORACLE 2

This audit was conducted for Atlas Oracle to evaluate the security and correctness of the smart contracts associated with the Atlas Oracle 2 project. The assessment included a comprehensive review of the in-scope smart contracts. The audit was performed using a combination of Formal Verification and Static Analysis.

The review process emphasized the following areas:

- Architecture review and threat modeling to understand systemic risks and identify design-level flaws.
- Identification of vulnerabilities through both common and edge-case attack vectors.
- Manual verification of contract logic to ensure alignment with intended design and business requirements.
- Dynamic testing to validate runtime behavior and assess execution risks.
- Assessment of code quality and maintainability, including adherence to current best practices and industry standards.

The audit resulted in findings categorized across multiple severity levels, from informational to critical. To enhance the project's security and long-term robustness, we recommend addressing the identified issues and considering the following general improvements:

- Improve code readability and maintainability by adopting a clean architectural pattern and modular design.
- Strengthen testing coverage, including unit and integration tests for key functionalities and edge cases.
- Maintain meaningful inline comments and documentations.
- Implement clear and transparent documentation for privileged roles and sensitive protocol operations.
- Regularly review and simulate contract behavior against newly emerging attack vectors.

REVIEW NOTES | ATLAS ORACLE 2

Out-of-Scope Components

1. The contract uses extra data provided by an off-chain SDK to decode prices and aggregatedTimestamps. As the off-chain system is currently out of scope, we assume the provided extra data is safe.
2. Most of the contract's API functions are internal. How external contracts inherit them and implement the corresponding functionality is currently out of scope; therefore, we assume the implementations in the external contracts are safe.

Centralization Related Risk

In the contract `PullOracleConsumerBase`, the role `authorized signer defined by the implementation` has authority over the following functions:

- `_getVerifiedFeedData()`
- `_getVerifiedFeedDataBatch()`
- `_getVerifiedFeedDataLenient()`
- `_getVerifiedFeedDataBatchLenient()`
- `_authenticateAndUnpackExtraData()`

Any compromise to the `authorized signer` defined by the implementation account may allow an attacker to take advantage of this authority and submit signed oracle payloads that are accepted by downstream consumers, potentially manipulating verified feed prices and timestamps used by protocol logic.

In the contract `PullOracleConsumerStandard`, the role `hardcoded authorized signer` has authority over the following functions:

- `_isAuthorizedSigner()`

Any compromise to the `hardcoded authorized signer` account may allow an attacker to take advantage of this authority and produce signed oracle payloads that pass the default signer check, potentially causing all consumers using this implementation to accept attacker-controlled price data. Because the signer address is hardcoded, signer rotation requires replacing or redeploying the consuming implementation.

In the contract `PullOracleConsumerTransient`, the role `hardcoded authorized signer` has authority over the following functions:

- `_isAuthorizedSigner()`

Any compromise to the `hardcoded authorized signer` account may allow an attacker to take advantage of this authority and produce signed oracle payloads that pass the default signer check, potentially causing all consumers using this implementation to accept attacker-controlled price data. Because the signer address is hardcoded, signer rotation requires replacing or redeploying the consuming implementation.

In the contract `PullOracleConsumerStandardStorage`, the role `authorized signer` has authority over the following functions:

- `_isAuthorizedSigner()`

Any compromise to the `authorized signer` account may allow an attacker to take advantage of this authority and submit signed oracle payloads that pass storage-backed signer validation, potentially manipulating price data accepted by consumer contracts.

FINDINGS | ATLAS ORACLE 2



8
Total Findings

0
Critical

0
Centralization

0
Major

2
Medium

4
Minor

2
Informational

This report has been prepared for Atlas Oracle to identify potential vulnerabilities and security issues within the reviewed codebase. During the course of the audit, a total of 8 issues were identified. Leveraging a combination of Formal Verification & Static Analysis the following findings were uncovered:

ID	Title	Category	Severity	Status
AO2-01	Inherited Standard Consumer Relies On A Single Hardcoded Oracle Signer	Logical Issue	Medium	● Acknowledged
AO2-02	Potential Dirty Data In <code>signer</code> Reading	Logical Issue	Medium	● Resolved
AO2-04	Unupdated Free Memory Pointer	Logical Issue	Minor	● Resolved
AO2-05	Duplicate Feed IDs Are Accepted And Can Resolve To Conflicting Oracle Values	Inconsistency, Logical Issue	Minor	● Acknowledged
AO2-06	Zero Price Values Are Accepted As Valid Oracle Data	Logical Issue, Inconsistency	Minor	● Acknowledged
AO2-07	Unchecked <code>feedIds</code>	Logical Issue	Minor	● Resolved
AO2-03	Empty Initial Signer Set Can Disable Oracle Consumption	Logical Issue	Informational	● Acknowledged
AO2-08	Clarification On Price Feed Extra Data Replay And Subscriber Access Control	Design Issue	Informational	● Acknowledged

AO2-01 | Inherited Standard Consumer Relies On A Single Hardcoded Oracle Signer

Category	Severity	Location	Status
Logical Issue	● Medium	projects/pull-oracle-consumer-tech-certik_audit_20260518/src-advanced/PullOracleConsumerTransient.sol (init-update-139188): 381; projects/pull-oracle-consumer-tech-certik_audit_20260518/src/PullOracleConsumerStandard.sol (init-update-139188): 39; projects/pull-oracle-consumer-tech-certik_audit_20260518/src/base/PullOracleConsumerBase.sol (init-update-139188): 449; projects/pull-oracle-consumer-tech-certik_audit_20260518/src/libraries/PullOracleDefaultHooks.sol (init-update-139188): 64	● Acknowledged

Description

As the business document says the SDK is designed as an abstract base class system where integrator contracts inherit `PullOracleConsumerStandard` or `PullOracleConsumerTransient` to gain internal Pull Oracle verification APIs. However, when a developer inherits `PullOracleConsumerStandard`, **the inherited signer authorization logic is not configurable by default.**

In `src/PullOracleConsumerStandard.sol`, `_isAuthorizedSigner()` delegates signer validation to `PullOracleDefaultHooks.isAuthorizedSigner()`:

```
function _isAuthorizedSigner(address recoveredSigner) internal view virtual override
returns (bool) {
    return PullOracleDefaultHooks.isAuthorizedSigner(recoveredSigner);
}
```

The default hook then accepts only a single hardcoded signer address:

```
return recoveredSigner == 0x59eD4701224fD9e2a85Ef2946c2ab828C1dDC600;
```

Because `_authenticateAndUnpackExtraData()` in the base contract rejects any payload not signed by `_isAuthorizedSigner()`, every downstream business contract that inherits `PullOracleConsumerStandard` without overriding this hook becomes fully dependent on this single signer.

If the hardcoded signer is compromised, an attacker may be able to submit malicious oracle payloads that pass verification and influence downstream business logic using manipulated prices or timestamps. Conversely, if the signer is lost, unavailable, revoked, or needs emergency rotation, integrator contracts using the default standard implementation have no on-chain mechanism to update the trusted signer. This creates both a centralization risk and an integration footgun for developers who may reasonably expect the SDK's standard consumer to be production-ready without overriding signer authorization.

Recommendation

Avoid making `PullOracleConsumerStandard` silently depend on a single hardcoded signer for production integrations.

Recommended mitigations include:

- Clearly document that `PullOracleConsumerStandard` uses a fixed signer and that integrators must override `_isAuthorizedSigner()` if they require configurable trust assumptions.
- Prefer a storage-backed signer registry for production deployments, such as the pattern used by `PullOracleConsumerStandardStorage`.
- Support signer rotation through owner, governance, or multisig-controlled update functions with appropriate events and timelock controls.

Alleviation

[Atlas Oracle, 05/26/2026]: The team acknowledged the issue and decided not to implement the recommended change in the current engagement but added detailed reminders and descriptions to the natspecs of the code(version 20260526), and will update the documentation in future to highlight the advantages and disadvantages of these two inheritance methods and provide guidance to users.

`PullOracleConsumerStandard.sol`

```
13  * NOTE: All hook parameters (authorized signer, timestamp boundaries,
14  * max package count) are
15  * hardcoded constants in PullOracleDefaultHooks,
16  * yielding the lowest possible gas consumption.
17  * The tradeoff is immutability – modifying any of these values post-
18  * deployment requires a
19  * contract upgrade. If runtime configurability is
20  * preferred over gas optimization, inherit
21  * from PullOracleConsumerStandardStorage instead, which provides a storage-
22  * backed registry
23  * with governance-controlled signer rotation, adjustable timestamp thresholds,
24  * and dynamic
25  * package count limits.
26  */
```

AO2-02 | Potential Dirty Data In `signer` Reading

Category	Severity	Location	Status
Logical Issue	Medium	projects/pull-oracle-consumer-tech-certik_audit_20260518/src/libraries/PullOracleSignature.sol (init-update-139188): 59~66	Resolved

Description

When `_recoverSigner` calls the `ecrecover` precompiled contract at address `0x01`, it stores the return value in the 32 bytes starting at memory offset `0x00`, and then reads this value as `signer` to return.

`PullOracleSignature.sol`

```
59         if iszero(staticcall(gas(), ECRECOVER_PRECOMPILE_ADDR, ptr, 128,
0x00, 32)) {
60             mstore(0x00, PRECOMPILE_CALL_FAILED_SELECTOR)
61             revert(0x00, 0x04)
62         }
63
64         // Retrieve the recovered address from the scratch space
65         // Defer zero address check to consumer layer to minimize gas
66         signer := mload(0x00)
```

However, when `ecrecover` runs out of gas or the signature recovery fails, it returns return data with length 0. Details can be found in the description of the [0x01 precompiled contract](#) on [evm.codes](#).

As a result, when an error occurs, `ecrecover` returns successfully but with empty return data. Meanwhile, `staticcall` performs a best-effort overwrite when copying return data into memory rather than fully clearing the target region. In other words, if an error occurs and the return data length is 0, the 32 bytes starting at `0x00` will retain their original contents.

This behavior can be confirmed from the EVM source code:

[instructions.go](#)

```

826 func opStaticCall(pc *uint64, evm *EVM, scope *ScopeContext) ([]byte, error)
{
827     // Pop gas. The actual gas is in evm.callGasTemp.
828     stack := scope.Stack
829     // We use it as a temporary value
830     temp := stack.pop()
831     gas := evm.callGasTemp
832     // Pop other call parameters.
833     addr, inOffset, inSize, retOffset, retSize := stack.pop(), stack.pop(),
stack.pop(), stack.pop(), stack.pop()
834     toAddr := common.Address(addr.Bytes20())
835     // Get arguments from the memory.
836     args := scope.Memory.GetPtr(inOffset.Uint64(), inSize.Uint64())
837
838     ret, returnGas, err := evm.StaticCall(scope.Contract.Address(), toAddr,
args, NewGasBudget(gas))
839     if err != nil {
840         temp.Clear()
841     } else {
842         temp.SetOne()
843     }
844     stack.push(&temp)
845     if err == nil || err == ErrExecutionReverted {
846 @>     scope.Memory.Set(retOffset.Uint64(), retSize.Uint64(), ret)
847     }
848
849     scope.Contract.RefundGas(returnGas, evm.Config.Tracer, tracing.
GasChangeCallLeftOverRefunded)
850
851     evm.returnData = ret
852     return ret, nil
853 }

```

Here, `scope.Memory.Set(retOffset.Uint64(), retSize.Uint64(), ret)` is used to copy `ret` into memory.

memory.go

```

55 func (m *Memory) Set(offset, size uint64, value []byte) {
56
57 // It's possible the offset is greater than 0 and size equals 0. This is because
58 // the calcMemSize (common.go) could potentially return 0 when size is zero (NO-OP)
59 if size > 0 {
60     // length of store may never be less than offset + size.
61     // The store should be resized PRIOR to setting the memory
62     if offset+size > uint64(len(m.store)) {
63         panic("invalid memory: store empty")
64     }
65 @> copy(m.store[offset:offset+size], value)
66 }

```

Inside `Memory.Set()`, `value` is copied into `m.store[offset:offset+size]`. The `copy` operation is Go's built-in `copy` function, which only copies up to the minimum length of the two slices. See [Built-in functions](#) for details. Therefore, when `value` has length 0, no copy occurs at all.

As a consequence, when an error occurs, the stale dirty data originally stored in the 32 bytes starting at `0x00` will be returned as `signer` instead of a zero address, potentially causing unexpected behavior in the program.

Recommendation

We recommend checking the size of `returndata` before returning the result.

Alleviation

[Atlas Oracle, 05/26/2026]: The team heeded the advice and resolved the issue by adding `returndata` zero size check.

`PullOracleSignature.sol`

```
59         if iszero(staticcall(gas(), ECRECOVER_PRECOMPILE_ADDR, ptr, 128,
0x00, 32)) {
60             mstore(0x00, PRECOMPILE_CALL_FAILED_SELECTOR)
61             revert(0x00, 0x04)
62         }
63
64
65     // Revert if ecrecover returned empty data (invalid v/r/s parameters)
66     if iszero(returndatasize()) {
67         mstore(0x00, SIGNATURE_RECOVERY_FAILED_SELECTOR)
68         revert(0x00, 0x04)
69     }
70
71     // Retrieve the recovered address from the scratch space
72     signer := mload(0x00)
```

AO2-04 | Updated Free Memory Pointer

Category	Severity	Location	Status
Logical Issue	Minor	projects/pull-oracle-consumer-tech-certik_audit_20260518/src/libraries/PullOracleSignature.sol (init-update-139188): 40-55	Resolved

Description

In `_recoverSigner()`, the contract reads `0x40`, the free memory pointer, as `ptr` to store data in the next available memory space.

`PullOracleSignature.sol`

```

40 @>         let ptr := mload(0x40)
41
42
43 // Calculate the total length of the signed data: packages + 1 byte for count
44         let signedLen := sub(sigOffset, payloadStart)
45
46 // Copy the raw signed payload [Packages][Count] directly to memory for hashing
47 @>         calldatacopy(ptr, payloadStart, signedLen)
48
49
50 // Omitting EIP-191 prefix reduces computational burden on the service and minimizes
51 // gas consumption for contract recovery
52
53
54 @>         mstore(ptr, keccak256(ptr, signedLen))
55
56
57 // Copy 'r' and 's' (64 bytes) directly from calldata to the ecrecover buffer
58
59
60 // This reduces stack pressure by avoiding separate let variables for 'r' and 's'
61 @>         calldatacopy(add(ptr, 64), sigOffset, 64)
62
63
64 // Populate 'v' to finalize ecrecover input layout: [hash][v][r][s]
65 @>         mstore(add(ptr, 32), byte(0, calldataload(add(sigOffset, 64))))

```

As a result, the memory region `mem[ptr:ptr + max(signedLen, 128)]` contains data. However, at the end of the function, the value at `0x40` is not updated to `ptr + max(signedLen, 128)`. Consequently, subsequent memory writes will start from the `ptr` position, which still contains stale data. This can potentially cause the newly written memory to include dirty data.

Even though the comment says “Skip updating 0x40 to save mstore gas”, considering that this is a library function and users may reuse it in their own contracts, we recommend not saving gas by skipping the update of the free memory pointer.

```

69          // Skip updating 0x40 to save mstore gas
70
// Permit subsequent logic to reclaim ephemeral memory starting from ptr

```

Recommendation

We recommend updating the free memory pointer at `0x40` as following:

```

let size := signedLen
if lt(size, 128) {
    size := 128
}
mstore(0x40, add(ptr, and(add(size, 31), not(31))))

```

Alleviation

[Atlas Oracle, 05/26/2026]: The team acknowledged the issue and decided not to implement the recommended change in the current engagement because the development team believes this issue would not occur under a reasonable development process.

[Atlas Oracle, 05/28/2026]:

The team heeded the advice and partially resolved the issue by updating the free memory pointer at the end of the `_recoverSigner` function and declaring the assembly block as `memory-safe`.

PullOracleSignature.sol

```

73
// Update the free memory pointer to cover the larger of signedLen and 128
(ecrecover input size)

74     let size := signedLen
75     if lt(signedLen, 128) {
76         size := 128
77     }
78     mstore(0x40, add(ptr, size))

```

[CertiK, 05/29/2026]:

`PullOracleSignature._recoverSigner()` updates the free memory pointer using `signedLen` without rounding it up to a 32-byte boundary. Since `signedLen = count * FEED_PACKAGE_SIZE + 1 = count * 20 + 1`, when the package count is at least 7, the function stores a non-32-byte-aligned address into `0x40`.

This polluted free memory pointer is later consumed by batch query functions such as

`_getAndValidateFeedValuesByIdsFromExtraDataOrRevertIfAnyUnmatched()` and `_getAndValidateFeedValuesByIdsFromExtraDataOrZeroIfUnmatched()`, which read `mload(0x40)` and manually allocate the `prices` and `aggregatedTimestamps` arrays from that address.

As a result, returned dynamic arrays and subsequent memory allocations may start from a non-32-byte-aligned address,

breaking Solidity's memory allocation convention and potentially causing inconsistent or unsafe memory layout.

It is recommended to round the allocated size up to the nearest 32-byte boundary before updating the free memory pointer.

```
let size := signedLen
if lt(size, 128) {
    size := 128
}
mstore(0x40, add(ptr, and(add(size, 31), not(31))))
```

[Atlas Oracle, 06/01/2026]:

The team heeded the advice and resolved the issue by rounding the allocated memory size up to the nearest 32-byte boundary.

```
73
// Update the free memory pointer to cover the larger of signedLen and 128
(ecrecover input size),

74
// rounded up to a 32-byte boundary to maintain Solidity's memory alignment
invariant.

75     let size := signedLen
76     if lt(signedLen, 128) {
77         size := 128
78     }
79     mstore(0x40, add(ptr, and(add(size, 31), not(31))))
```


AO2-05 Duplicate Feed IDs Are Accepted And Can Resolve To Conflicting Oracle Values

Category	Severity	Location	Status
Inconsistency, Logical Issue	Minor	projects/pull-oracle-consumer-tech-certik_audit_20260518/src-advanced/PullOracleConsumerTransient.sol (init-update-139188): 314; projects/pull-oracle-consumer-tech-certik_audit_20260518/src/base/PullOracleConsumerBase.sol (init-update-139188): 123, 123, 175, 288, 334	Acknowledged

Description

The oracle payload parser authenticates the signed package list, count, and signer, but it does not enforce that each feed ID appears only once. The base lookup functions in `PullOracleConsumerBase` scan the signed payload and stop at the first package whose high 4 bytes match the requested feed ID. If a validly signed payload contains the same feed ID multiple times with different prices or timestamps, the base strict and lenient single/batch paths silently accept the payload and return the first matching value while ignoring later conflicting values.

The transient batch implementation has the same missing uniqueness invariant but resolves the conflict differently. It builds an $O(1)$ lookup table by storing each signed feed package in transient storage under `and(word, FEED_ID_MASK)`, so each later `tstore` overwrites the earlier value for the same feed ID. As a result, transient batch lookups resolve duplicate feed IDs using last-write-wins semantics, while the base linear-scan paths resolve the same duplicated payload using first-match-wins semantics.

Although the documentation states that for the same feed ID, the aggregate parameters and asset pair are the same—meaning the price is unique—this still represents a protocol-level ambiguity and a centralization/integrity risk: a malicious, compromised, or faulty authorized signer can publish one signed payload containing conflicting entries, causing consumers to accept ambiguous oracle data and, depending on the accessor path, observe different verified values while all calls pass signature verification. This can break cross-contract assumptions and make incident analysis harder because there is no canonical value for a duplicated feed ID within one signed payload.

Recommendation

Consider rejecting signed payloads that contain duplicate feed IDs during payload validation or before returning/caching any package, so every valid payload has a single canonical package per feed ID. Alternatively, consider defining and enforcing one duplicate-resolution rule consistently across the base and transient implementations, and document that rule explicitly for integrators.

In addition to the `feedId`s in the `packages` within calldata being allowed to duplicate, the `feedIds[]` array passed into the function is also permitted to contain duplicates. We recommend that the team indicate in the function comments that no duplicate checks are performed, to prevent misuse.

I Alleviation

[Atlas Oracle, 05/26/2026]: The team acknowledged the issue, decided not to implement the recommended change in the current engagement but added explanatory notes regarding duplicate feedId values in the input parameters for the following four user-callable batch API functions(version 20260526), and provided the following statement:

Our backend API validates the extra package list to ensure no duplicate feedIds are present. As a result, the contract-side code assumes that feedIds in the package list are unique. Adding duplicate feedId validation on the contract side for the package list would require $O(n^2)$ comparisons or sorting on calldata, incurring substantial gas overhead—contradicting the project's "extreme gas optimization" design.

AO2-06 | Zero Price Values Are Accepted As Valid Oracle Data

Category	Severity	Location	Status
Logical Issue, Inconsistency	Minor	projects/pull-oracle-consumer-tech-certik_audit_20260518/src-advanced/PullOracleConsumerTransient.sol (init-update-139188): 172, 276; projects/pull-oracle-consumer-tech-certik_audit_20260518/src/base/PullOracleConsumerBase.sol (init-update-139188): 155, 253, 316, 406	Acknowledged

Description

`PullOracleConsumerBase` / `PullOracleConsumerTransient` decodes each matched feed package with `word._parseFeedPackage()` and then validates only the aggregated timestamp through `_validateTimestamp`. The decoded `price` is returned to the caller without checking whether it is non-zero or otherwise within a valid range.

For price oracle feeds, a zero price is generally an invalid market value. If an authorized signer, a compromised signer, or an off-chain aggregation fault produces a package with `price == 0` and a fresh timestamp, the strict getters will accept it as verified data. Downstream integrations may then value collateral at zero, trigger incorrect liquidations, divide by zero, or otherwise execute business logic using an invalid oracle value.

The issue is also inconsistent with lenient mode semantics. Lenient getters use zeroed return values to indicate that a requested feed ID was not found in the signed payload. Because a present feed with a signed zero price is also accepted, consumers cannot reliably distinguish an absent feed from a present-but-zero-priced feed unless they add their own additional checks around the returned timestamp and price.

This is not a permissionless manipulation path because producing a fresh signed zero-price package requires an authorized signer or signer compromise. However, the current base implementation lacks a fundamental sanity check for oracle data and expands the blast radius of signer or off-chain data failures.

Recommendation

Consider rejecting decoded feed packages when `price == 0` before returning the value to callers. Consider adding a dedicated `_validatePrice(bytes4 feedId, uint256 price)` hook, called alongside `_validateTimestamp`, so inheriting consumers can enforce non-zero prices or feed-specific bounds consistently across strict and lenient single/batch paths.

Alleviation

[Atlas Oracle, 05/26/2026]: The team acknowledged the issue and decided not to implement the recommended change in the current engagement, and provided the following statement:

Price reasonableness falls under the business logic of the consumer side and is outside the scope of the protocol's verification layer. Similar to a push oracle: if the price actually becomes zero (an extreme case), the user should handle this check within their own business contract, while the oracle is only responsible for delivering the real-world truth. If a

new hook like `_validatePrice` is added at the consumer layer, integrators might mistakenly assume that "the oracle has already validated the price for me," reducing their awareness of performing boundary checks on their own.

AO2-07 | Unchecked `feedIds`

Category	Severity	Location	Status
Logical Issue	Minor	projects/pull-oracle-consumer-tech-certik_audit_20260518/src-advanced/ PullOracleConsumerTransient.sol (init-update-139188): 157~160, 269~272	Resolved

Description

In `_getAndValidateFeedValuesByIdsFromExtraDataOrRevertIfAnyUnmatchedTransient()` and `_getAndValidateFeedValuesByIdsFromExtraDataOrZeroIfUnmatchedTransient()`, a default assumption is made: transient storage is completely empty before this function is called. Under this assumption, values read by `tload()` are guaranteed to have been written only by `_cacheFeedPackagesToTransientStorage(payloadStart, payloadEnd)`.

However, there is no guarantee that this assumption holds. Within a single transaction, transient storage could potentially be written with other data. In such a case, if `feedIds` contains IDs not present in `packages`, and the transient key corresponding to this ID has been written with other data within the same transaction, `tload` will read that data. This can result in unverified price and aggregated timestamp being used.

Recommendation

We recommend ensuring that all elements of `feedIds` correspond to empty entries in transient storage before calling `_cacheFeedPackagesToTransientStorage()`. In other words, clear the transient storage values for the keys corresponding to `feedIds` prior to `_cacheFeedPackagesToTransientStorage()`.

Alleviation

[Atlas Oracle, 05/28/2026]:

The team heeded the advice and resolved the issue by the following updates:

1. After reading from transient storage, the implementation now performs a lightweight validation to ensure that the high 4 bytes of the loaded `word` match the requested `targetFeedId`. If the check passes, the word is treated as a valid cached package. Otherwise, strict mode reverts as unmatched, while lenient mode returns zero values.

```
// Validate: a non-zero word must belong to the current signed payload.
// Legitimate cache entries have the feed ID in the high 4 bytes.
// If the high 4 bytes do not match, the slot contains foreign
// transient data written by other logic – treat as unmatched.
if iszero(eq(and(word, FEED_ID_MASK), targetFeedId)) {
    word := 0
}
```

2. Security warnings were added to the contract-level NatSpec of `PullOracleConsumerTransient` and `PullOracleConsumerTransientStorage`, as well as to the NatSpec of `_getVerifiedFeedDataBatchTransient()` and

`_getVerifiedFeedDataBatchLenientTransient()`, to document the reserved feed-ID-keyed transient storage slots and warn inheriting contracts not to perform conflicting `TSTORE` operations within the same transaction.

[Certik, 05/29/2026]:

This resolution relies on the assumption that inheriting or upper-layer contracts do not intentionally or unexpectedly write attacker-controlled values to the reserved feed-ID-keyed transient storage slots. If such writes are still possible, a value whose high 4 bytes match the requested `targetFeedId` may still be interpreted as a valid cached oracle package.

AO2-03 | Empty Initial Signer Set Can Disable Oracle Consumption

Category	Severity	Location	Status
Logical Issue	● Informational	projects/pull-oracle-consumer-tech-certik_audit_20260518/src-advanced/PullOracleConsumerTransientStorage.sol (init-update-139188): 65, 123; projects/pull-oracle-consumer-tech-certik_audit_20260518/src/PullOracleConsumerStandardStorage.sol (init-update-139188): 66, 125; projects/pull-oracle-consumer-tech-certik_audit_20260518/src/base/PullOracleConsumerBase.sol (init-update-139188): 449	● Acknowledged

Description

Both the `PullOracleConsumerStandardStorage` constructor and the `PullOracleConsumerTransientStorage` constructor reads `initialSigners.length` and iterates over the provided signer list, but they do not reject an empty `initialSigners` array. Because signer authorization is a key state variable for the oracle consumer, deploying with no authorized signer leaves the contract unable to accept any signed oracle payload.

This does not create an arbitrary-signer bypass: `PullOracleConsumerBase` still recovers the signer from the payload and reverts when `_isAuthorizedSigner` returns false. However, the fail-closed behavior means a deployer mistake, deployment script bug, or incorrectly populated constructor argument can create a consumer that is operationally useless until a signer is added through a correctly exposed and access-controlled setter.

The impact depends on the concrete inheriting contract. If the child contract exposes governed signer management, the issue is a deployment-time availability and misconfiguration risk. If the child contract does not expose any post-deployment signer update function, the deployment can be permanently unable to consume oracle data and may require redeployment.

Recommendation

Consider rejecting empty `initialSigners` in the constructor by reverting when `initialSigners.length == 0`, or explicitly implementing and documenting a safe two-step deployment flow that requires an authorized signer to be configured before any oracle-dependent functionality can be used.

Alleviation

[Atlas Oracle, 05/26/2026]: The team acknowledged the issue and decided not to implement the recommended change in the current engagement but added detailed reminders and descriptions to the natspecs of the code(version 20260526).

`PullOracleConsumerStandardStorage.sol`

```
52      * @dev Initialize the packed config slot and seed the authorized signer set
.
53      *      An empty initialSigners array is
intentionally permitted to support two-step
54      *
      deployment flows where signers are added via governance after deployment.
55      *      If initialSigners is empty,
the inheritor MUST expose _setSignerStatus through
56      *      an access-controlled public setter; otherwise the contract will
be permanently
57      *      unable to verify any oracle payload.
```


AO2-08 Clarification On Price Feed Extra Data Replay And Subscriber Access Control

Category	Severity	Location	Status
Design Issue	● Informational	projects/pull-oracle-consumer-tech-certik_audit_20260518/src-advanced/PullOracleConsumerTransient.sol (init-update-139188): 15; projects/pull-oracle-consumer-tech-certik_audit_20260518/src-advanced/PullOracleConsumerTransientStorage.sol (init-update-139188): 32; projects/pull-oracle-consumer-tech-certik_audit_20260518/src/PullOracleConsumerStandard.sol (init-update-139188): 12; projects/pull-oracle-consumer-tech-certik_audit_20260518/src/PullOracleConsumerStandardStorage.sol (init-update-139188): 33; projects/pull-oracle-consumer-tech-certik_audit_20260518/src/base/PullOracleConsumerBase.sol (init-update-139188): 15	● Acknowledged

Description

The Pull Oracle consumer SDK follows a stateless pull model: a backend or SDK supplies trailing `calldata` extra data, and internal APIs such as `_getVerifiedFeedData` validate that suffix during execution. On-chain checks are limited to layout and magic-marker parsing, recovery of an authorized oracle signer over `keccak256(packages || count)`, and freshness bounds via `_validateTimestamp`. The signed material does not include `msg.sender`, or a per-use nonce.

Since fetching extra data off-chain and submitting a transaction on-chain are separate steps. The same blob can be reused by any address while package timestamps remain within `maxDelay` and `maxFutureDrift`. Because extra data is public in transaction `calldata`, observers can copy it from the mempool or explorers. In the stronger case, a third party can replay **byte-identical** `msg.data` to the **same** integrator contract—same selector, same business arguments, same oracle suffix—with only `msg.sender` changed; the on-chain contracts will accept the suffix again on each execution until it goes stale. At the same time, within the same time window, multiple valid prices may exist within the freshness window. Observers can selectively choose the most favorable price as input. If the price oracle is used for staking or lending projects, this could potentially be exploited for profit.

Recommendation

We would like to check with the team whether “subscriber-only” price access is meant to be enforced only off-chain or in integrator business logic, whether replay of the same `calldata`—including full transaction input replay to the same contract within the freshness window—is acceptable by design and the behavior of users/observers selecting the most favorable price within a given time window is acceptable.

Alleviation

[Atlas Oracle, 05/28/2026]:

The team acknowledged the issue and decided not to implement the recommended change in the current engagement, and provided the following statement:

Subscribers to our service can obtain the extra data off-chain through the TypeScript SDK and API key that we provide, and then append it to their on-chain transactions. However, we cannot control how users of our Pull Oracle integrate or use it within their own business logic.

Therefore, given the stateless design of the Pull Oracle, replaying the same calldata against the same contract within the freshness window is unavoidable. This is also why we provide the `validateTimestamp` hook for users to implement their own validation logic. If a customer is highly sensitive to price freshness, they should configure stricter timestamp validation thresholds.

APPENDIX | ATLAS ORACLE 2

Audit Scope

CertiKProject/certik-audit-projects

- projects/pull-oracle-consumer-tech-certik_audit_20260518/src-advanced/PullOracleConsumerTransient.sol
- projects/pull-oracle-consumer-tech-certik_audit_20260518/src-advanced/PullOracleConsumerTransientStorage.sol
- projects/pull-oracle-consumer-tech-certik_audit_20260518/src/base/PullOracleConsumerBase.sol
- projects/pull-oracle-consumer-tech-certik_audit_20260518/src/libraries/PullOracleDefaultHooks.sol
- projects/pull-oracle-consumer-tech-certik_audit_20260518/src/PullOracleConsumerStandard.sol
- projects/pull-oracle-consumer-tech-certik_audit_20260518/src/PullOracleConsumerStandardStorage.sol
- projects/pull-oracle-consumer-tech-certik_audit_20260518/src/libraries/PullOracleSignature.sol
- projects/pull-oracle-consumer-tech-certik_audit_20260518/src/constants/Constants.sol
- projects/pull-oracle-consumer-tech-certik_audit_20260518/src/interfaces/IPullOracleBase.sol
- projects/pull-oracle-consumer-tech-certik_audit_20260518/src/interfaces/IPullOracleConsumerStorage.sol
- projects/pull-oracle-consumer-tech-certik_audit_20260518/src/interfaces/IPullOracleDefaultHooks.sol
- projects/pull-oracle-consumer-tech-certik_audit_20260518/src/libraries/CustomErrorRevert.sol
- projects/pull-oracle-consumer-tech-certik_audit_20260518/src/libraries/PullOracleCodec.sol
- projects/pull-oracle-consumer-tech-certik_audit_20260518/src/libraries/PullOracleConfigCodec.sol
- projects/pull-oracle-consumer-tech-certik_audit_20260601/src-advanced/PullOracleConsumerTransient.sol
- projects/pull-oracle-consumer-tech-certik_audit_20260601/src-advanced/PullOracleConsumerTransientStorage.sol
- projects/pull-oracle-consumer-tech-certik_audit_20260601/src/base/PullOracleConsumerBase.sol
- projects/pull-oracle-consumer-tech-certik_audit_20260601/src/constants/Constants.sol

CertiKProject/certik-audit-projects

-  projects/pull-oracle-consumer-tech-certik_audit_20260601/src/interfaces/IPullOracleBase.sol
-  projects/pull-oracle-consumer-tech-certik_audit_20260601/src/interfaces/IPullOracleConsumerStorage.sol
-  projects/pull-oracle-consumer-tech-certik_audit_20260601/src/interfaces/IPullOracleReferenceHooks.sol
-  projects/pull-oracle-consumer-tech-certik_audit_20260601/src/libraries/CustomErrorRevert.sol
-  projects/pull-oracle-consumer-tech-certik_audit_20260601/src/libraries/PullOracleCodec.sol
-  projects/pull-oracle-consumer-tech-certik_audit_20260601/src/libraries/PullOracleConfigCodec.sol
-  projects/pull-oracle-consumer-tech-certik_audit_20260601/src/libraries/PullOracleReferenceHooks.sol
-  projects/pull-oracle-consumer-tech-certik_audit_20260601/src/libraries/PullOracleSignature.sol
-  projects/pull-oracle-consumer-tech-certik_audit_20260601/src/PullOracleConsumerStandard.sol
-  projects/pull-oracle-consumer-tech-certik_audit_20260601/src/PullOracleConsumerStandardStorage.sol

oracle-atlas/pull-oracle-consumer

-  src-advanced/PullOracleConsumerTransient.sol
-  src-advanced/PullOracleConsumerTransientStorage.sol
-  src/base/PullOracleConsumerBase.sol
-  src/constants/Constants.sol
-  src/interfaces/IPullOracleBase.sol
-  src/interfaces/IPullOracleConsumerStorage.sol
-  src/interfaces/IPullOracleReferenceHooks.sol
-  src/libraries/CustomErrorRevert.sol
-  src/libraries/PullOracleCodec.sol
-  src/libraries/PullOracleConfigCodec.sol

oracle-atlas/pull-oracle-consumer

 src/libraries/PullOracleReferenceHooks.sol

 src/libraries/PullOracleSignature.sol

 src/PullOracleConsumerStandard.sol

 src/PullOracleConsumerStandardStorage.sol

Finding Categories

Categories	Description
Logical Issue	Logical Issue findings indicate general implementation issues related to the program logic.
Inconsistency	Inconsistency findings refer to different parts of code that are not consistent or code that does not behave according to its specification.
Design Issue	Design Issue findings indicate general issues at the design level beyond program logic that are not covered by other finding categories.

DISCLAIMER | CERTIK

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability) set forth in the Services Agreement, or the scope of services, and terms and conditions provided to you ("Customer" or the "Company") in connection with the Agreement. This report provided in connection with the Services set forth in the Agreement shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes, nor may copies be delivered to any other person other than the Company, without CertiK's prior written consent in each instance.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts CertiK to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. CertiK's position is that each company and individual are responsible for their own due diligence and continuous security. CertiK's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by CertiK is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

ALL SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF ARE PROVIDED "AS IS" AND "AS AVAILABLE" AND WITH ALL FAULTS AND DEFECTS WITHOUT WARRANTY OF ANY KIND. TO THE MAXIMUM EXTENT PERMITTED UNDER APPLICABLE LAW, CERTIK HEREBY DISCLAIMS ALL WARRANTIES, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE WITH RESPECT TO THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS. WITHOUT LIMITING THE FOREGOING, CERTIK SPECIFICALLY DISCLAIMS ALL IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE AND NON-INFRINGEMENT, AND ALL WARRANTIES ARISING FROM COURSE OF DEALING, USAGE, OR TRADE PRACTICE. WITHOUT LIMITING THE FOREGOING, CERTIK MAKES NO WARRANTY OF ANY KIND THAT THE SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF, WILL MEET CUSTOMER'S OR ANY OTHER PERSON'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULT, BE COMPATIBLE OR WORK WITH ANY SOFTWARE, SYSTEM, OR OTHER SERVICES, OR BE SECURE, ACCURATE, COMPLETE, FREE OF HARMFUL CODE, OR ERROR-FREE. WITHOUT LIMITATION TO THE FOREGOING, CERTIK PROVIDES NO WARRANTY OR

UNDERTAKING, AND MAKES NO REPRESENTATION OF ANY KIND THAT THE SERVICE WILL MEET CUSTOMER'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULTS, BE COMPATIBLE OR WORK WITH ANY OTHER SOFTWARE, APPLICATIONS, SYSTEMS OR SERVICES, OPERATE WITHOUT INTERRUPTION, MEET ANY PERFORMANCE OR RELIABILITY STANDARDS OR BE ERROR FREE OR THAT ANY ERRORS OR DEFECTS CAN OR WILL BE CORRECTED.

WITHOUT LIMITING THE FOREGOING, NEITHER CERTIK NOR ANY OF CERTIK'S AGENTS MAKES ANY REPRESENTATION OR WARRANTY OF ANY KIND, EXPRESS OR IMPLIED AS TO THE ACCURACY, RELIABILITY, OR CURRENCY OF ANY INFORMATION OR CONTENT PROVIDED THROUGH THE SERVICE. CERTIK WILL ASSUME NO LIABILITY OR RESPONSIBILITY FOR (I) ANY ERRORS, MISTAKES, OR INACCURACIES OF CONTENT AND MATERIALS OR FOR ANY LOSS OR DAMAGE OF ANY KIND INCURRED AS A RESULT OF THE USE OF ANY CONTENT, OR (II) ANY PERSONAL INJURY OR PROPERTY DAMAGE, OF ANY NATURE WHATSOEVER, RESULTING FROM CUSTOMER'S ACCESS TO OR USE OF THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS.

ALL THIRD-PARTY MATERIALS ARE PROVIDED "AS IS" AND ANY REPRESENTATION OR WARRANTY OF OR CONCERNING ANY THIRD-PARTY MATERIALS IS STRICTLY BETWEEN CUSTOMER AND THE THIRD-PARTY OWNER OR DISTRIBUTOR OF THE THIRD-PARTY MATERIALS.

THE SERVICES, ASSESSMENT REPORT, AND ANY OTHER MATERIALS HEREUNDER ARE SOLELY PROVIDED TO CUSTOMER AND MAY NOT BE RELIED ON BY ANY OTHER PERSON OR FOR ANY PURPOSE NOT SPECIFICALLY IDENTIFIED IN THIS AGREEMENT, NOR MAY COPIES BE DELIVERED TO, ANY OTHER PERSON WITHOUT CERTIK'S PRIOR WRITTEN CONSENT IN EACH INSTANCE.

NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS.

THE REPRESENTATIONS AND WARRANTIES OF CERTIK CONTAINED IN THIS AGREEMENT ARE SOLELY FOR THE BENEFIT OF CUSTOMER. ACCORDINGLY, NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH REPRESENTATIONS AND WARRANTIES AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH REPRESENTATIONS OR WARRANTIES OR ANY MATTER SUBJECT TO OR RESULTING IN INDEMNIFICATION UNDER THIS AGREEMENT OR OTHERWISE.

FOR AVOIDANCE OF DOUBT, THE SERVICES, INCLUDING ANY ASSOCIATED ASSESSMENT REPORTS OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.

Elevate Your Web3 Journey

CertiK is the largest Web3 security platform combining formal verification with audits and comprehensive security solutions.

Atlas Oracle 2 Security Assessment | CertiK Assessed on Jun 5th, 2026 | Copyright © CertiK

